Pimpri Chinchwad Education Trust's

Pimpri Chinchwad College of Engineering and Research, Pune

Department of Computer Engineering

Subject: Operating System

Teaching Faculty:

Mrs. Shrutika A. Menkudale

Introduction to Syllabus

Course :- PCC-203- COM: Operating Systems

Teaching /scheme	Credits	Examination Scheme
Theory : 03 Hours/Week	03	CCE : 30 Marks End-Semester: 70 Marks

Prerequisite Courses : Data Structure, Digital Electronics

Companion Course : Computer Organization and Microprocessors

Course Objective

1. To **understand** the fundamental concepts, types, and structures of Operating Systems.

2. To **understand** and **analyze** process management concepts.

3. To **identify** and **solve** concurrency and deadlock in the operating system.

4. **Explore** the various techniques of memory management.

5. **Understand** I/O management, disk scheduling, and file systems

Course Outcome

- CO1: **Analyze** the fundamentals of Operating Systems, including types, structures, system calls, and basic Linux commands.

- CO2: **Apply** process scheduling and synchronization to optimize CPU utilization in modern operating systems.

- CO3: **Identify** the mechanism for dealing with deadlocks and concurrency concerns.

- CO4: **Apply** techniques of memory management to solve memory management problems.

- CO5: **Illustrate** I/O and file management policies.

Course Syllabus

Unit I- Introduction to Operating System (09 Hours)

Basics of Operating Systems: Objectives & Functions, Evolution of OS, Types of Operating Systems, OS Service, System Calls: Introduction, Types of System Calls.

OS Structure: Layered Approach, Monolithic, Microkernel Operating Systems.

Introduction to Linux OS: Components of Linux system, Basic Shell commands.

Case studies:

1. Automating User and File Management in Linux using Shell Script
2. Demonstrating Different OS Structures (Monolithic, Layered, Microkernel OS)

Course Syllabus

Unit II- Process and Thread Management (09 Hours)

Process management: Definition, types of process. Process States and Transitions diagram, Process Control Block (PCB), context switching and its impact on performance, Process Scheduling.

Types of Schedulers: long term, short term, middle term, Threads: Concept of thread, Multithreading, User level vs Kernel-level Threads.

Scheduling Algorithms: Preemptive Scheduling vs Non-preemptive Scheduling, FCFS, SJF, RR, Priority Process Scheduling in UNIX and Windows

Cast Studies-

1) Prepare case study on challenges for Real-time Scheduling
2)Performance Com parison of Scheduling Algorithms (FCFS vs SJF, RR vs Priority Scheduling) under different workload conditions

Course Syllabus

Unit III- Inter-process Communication and Deadlock (09 Hours)

Concurrency: Critical section problem, Synchronization primitives (Semaphores, Mutexes, Monitors)

Synchronization Problems: Producer-Consumer, Reader Writer, Dining Philosophers

Inter-Process Communication (IPC): Message passing, Shared memory Deadlocks: Conditions, Prevention, Avoid ance (Banker's Algorithm), Detection, Recovery

Case Studies-

1) Interprocess Communication (IPC) in a Banking System

2) Deadlock in Railway Scheduling Systems

Course Syllabus

Unit IV- Memory Management (09 Hours)

Introduction: Contiguous and non-contiguous.

Fragmentation: Internal and External fragmentation.

Memory allocation strategies: First Fit, Best Fit, Worst Fit, Memory **Partitioning:** Fixed and Dynamic partitions.

Paging: Structure of page table, Segmentation.Virtual Memory: Background, Demand Pag ing.

Page Replacement: FIFO, LRU, Optimal Thrashing.

Case study :

Intel Premium

Course Syllabus

Unit V- File and Disk Management (09 Hours)

File Management: File operation, Directory structure, File System structure, File Organization and Access, File Directories, File Allocation Methods, Secondary Storage Management, File Systems in Operating System (FAT, NTFS, EXT, and HFS).

Disk Management: Secondary-Storage Structure- Disk structure, Disk scheduling algorithm (FIFO, SSTF, SCAN, C-SCAN, LOOK, C-LOOK), Disk reliability, Disk formatting, Boot-block, Bad blocks.

Case Studies-

1) Study of Linux File System.
2) Study of Android File System.

Unit I
Introduction to Operating System

O
P
R
A
T
I
N
G

S
Y
S
T
E
M

How do I use them?

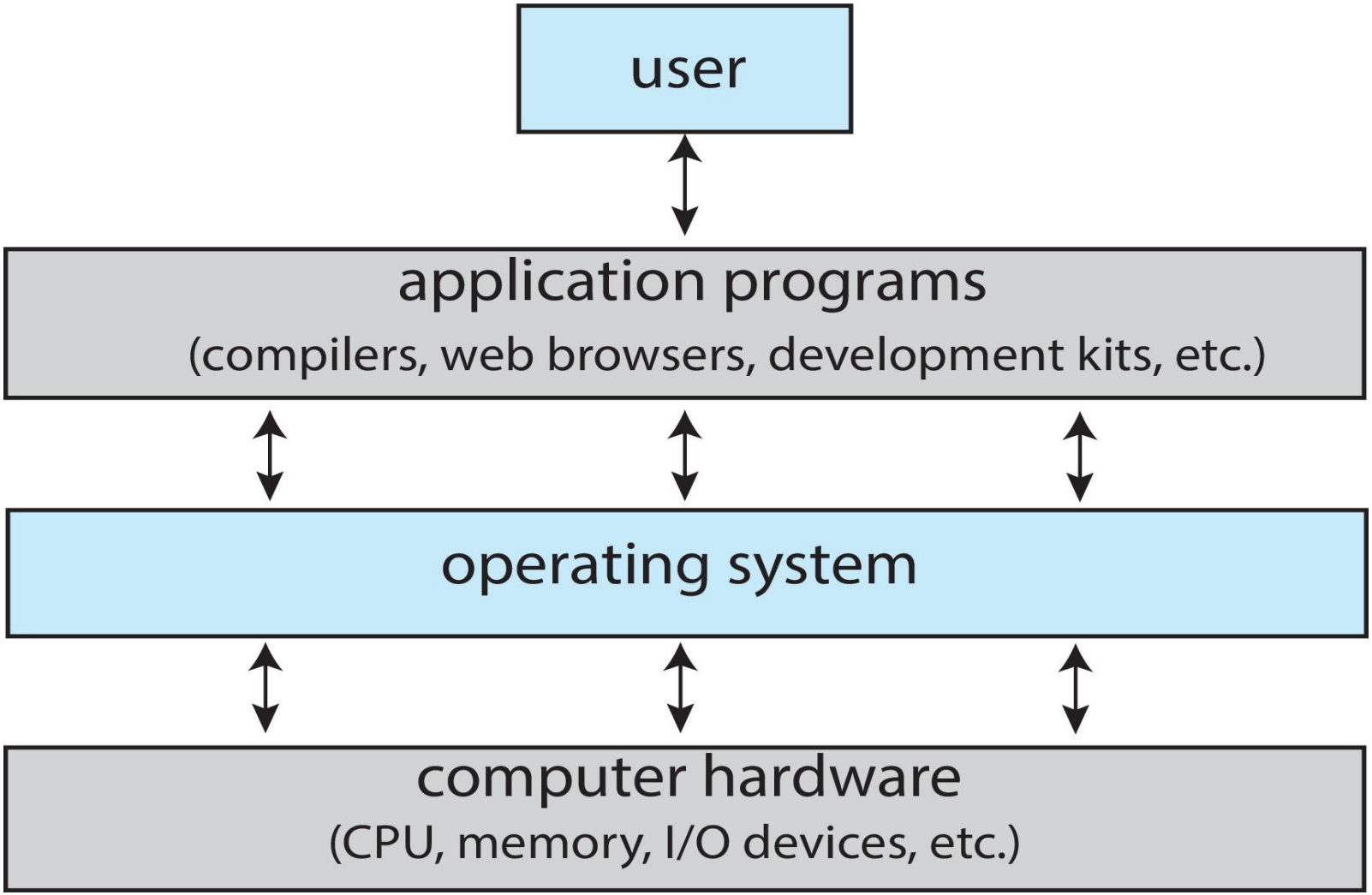
How do I manage them?

Unit I
Introduction to Operating System

- A program that acts as an <u>intermediary</u> between a user of a computer and the computer hardware.
- Operating system is large and complex, it must be created piece by piece.

- Operating system goals:
  - Execute user programs and make solving user problems easier
  - Make the computer system convenient to use
  - Use the computer hardware in an efficient manner

Abstract View of Components of Computer



Abstract View of Components of Computer

- Computer system can be divided into four components:
  - **Hardware** – provides basic computing resources
    - CPU, memory, I/O devices
  - **Operating system**
    - Controls and coordinates use of hardware among various applications and users
  - **Application programs** – define the ways in which the system resources are used to solve the computing problems of the users
    - Word processors, compilers, web browsers, database systems, video games
  - **Users**
    - People, machines, other computers

Objectives of an Operating System (OS)

Made with Napkin

Objectives of an Operating System (OS)

Resource Management

✔ Efficiently manages hardware resources like CPU, memory, disk, and I/O devices.

Convenience for Users

✔Provides an easy-to-use interface (GUI/CLI) so users can interact with the system without needing to understand hardware details.

Efficiency

✔Ensures maximum utilization of system resources for better performance.

Security and Access Control

✔Protects data and resources from unauthorized access.

Objectives of an Operating System (OS)

System Performance Monitoring

- ✔ Tracks performance and suggests improvements or adjustments.

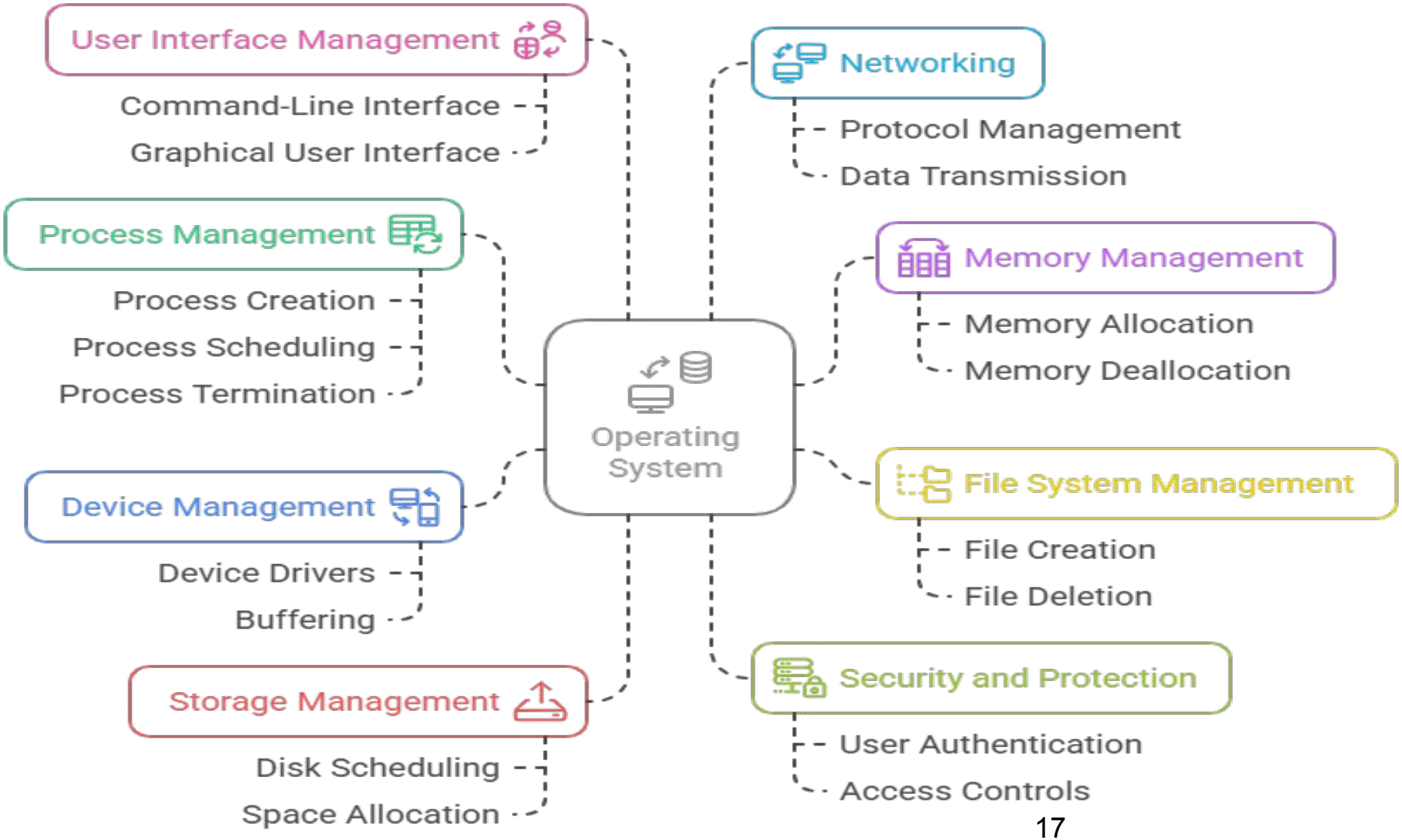
Error Detection and Handling

- ✔ Detects and handles errors at hardware and software levels.

Support for Multiprogramming

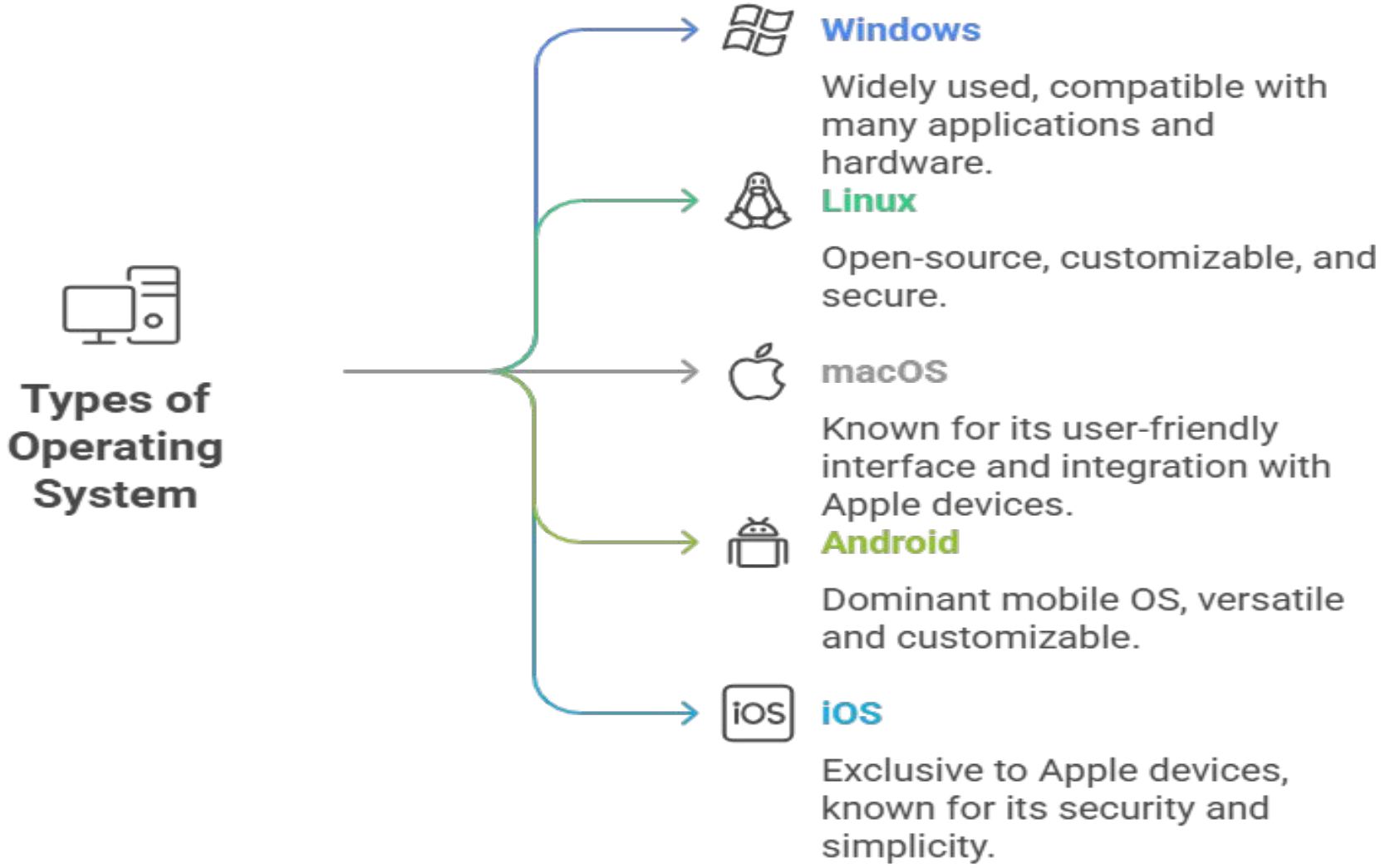
- ✔ Allows multiple programs to run simultaneously, increasing CPU utilization.

Function of an Operating System (OS)



Made with Napkin

Types of an Operating System (OS)



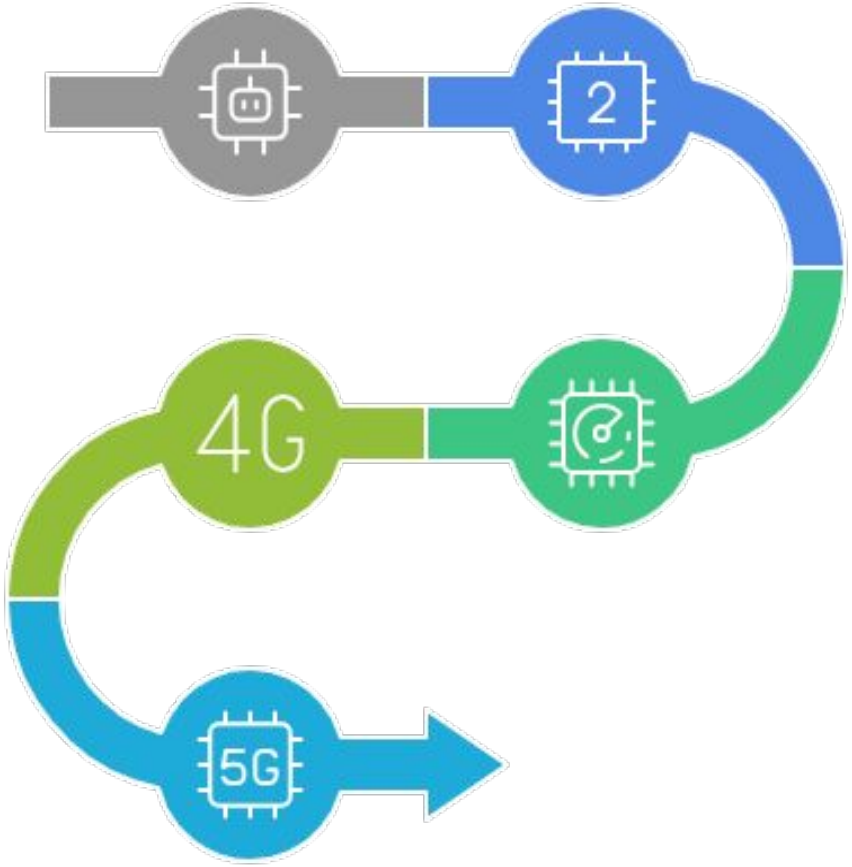
Made with Napkin

Evolution of an Operating System (OS)

- The evolution of operating systems can be understood through different **generations**, marked by technological advances in **hardware** and **user needs**.

Evolution of an Operating System (OS)

Evolution of Operating System



1940s–1950s

First Generation: Vacuum Tubes & Plugboards

1950s–1960s

Second Generation: Batch Processing Systems

1960s–1970s

Third Generation: Multiprogramming and Time Sharing

1980s–Present

Fourth Generation: GUI-based Personal Computers

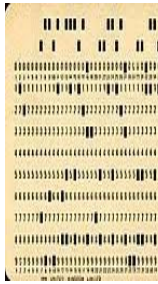
Present–Future

Fifth Generation: Networked, Distributed, and Mobile OS

Made with Napkin

First Generation (1940s–1950s): Vacuum Tubes & Plugboards

- **No OS** – programs executed directly using machine language.



- **Input/Output**: Manual via punch cards and paper tape.

- **Execution**: One job at a time; very slow and tedious.

Machines of the time were so primitive that programs were often entered <u>one bit at time</u> on rows of mechanical switches (plug boards).

- Programming languages were unknown (not even assembly languages).

Second Generation (1950s–1960s): Batch Processing Systems

- **OS Concept Emerged** – Simple batch operating systems.

- **Features**:
  - These were called single-stream batch processing systems because programs and data were submitted in groups or batches.
  - Input via punch cards, output on paper.
  - No user interaction during execution.

- **Example**: IBM 7094.

Third Generation (1960s–1970s): Multiprogramming and Time Sharing

- **Key Features**:
  - **Multiprogramming**: Multiple programs loaded in memory.
  - **Time Sharing:** Users interact via terminals, get CPU time in slices.
- **Introduction of**:
  - File systems
  - Spooling
  - Scheduling

- **Example**: UNIX (early versions), MULTICS.

Fourth Generation (1980s–Present): GUI-based Personal Computers

- **Main Developments**:
  - Graphical User Interface (GUI)
  - WIMP (Windows, Icons, Menus, Pointers)
  - Mouse and keyboard input

- **Examples**: Microsoft Windows, macOS.

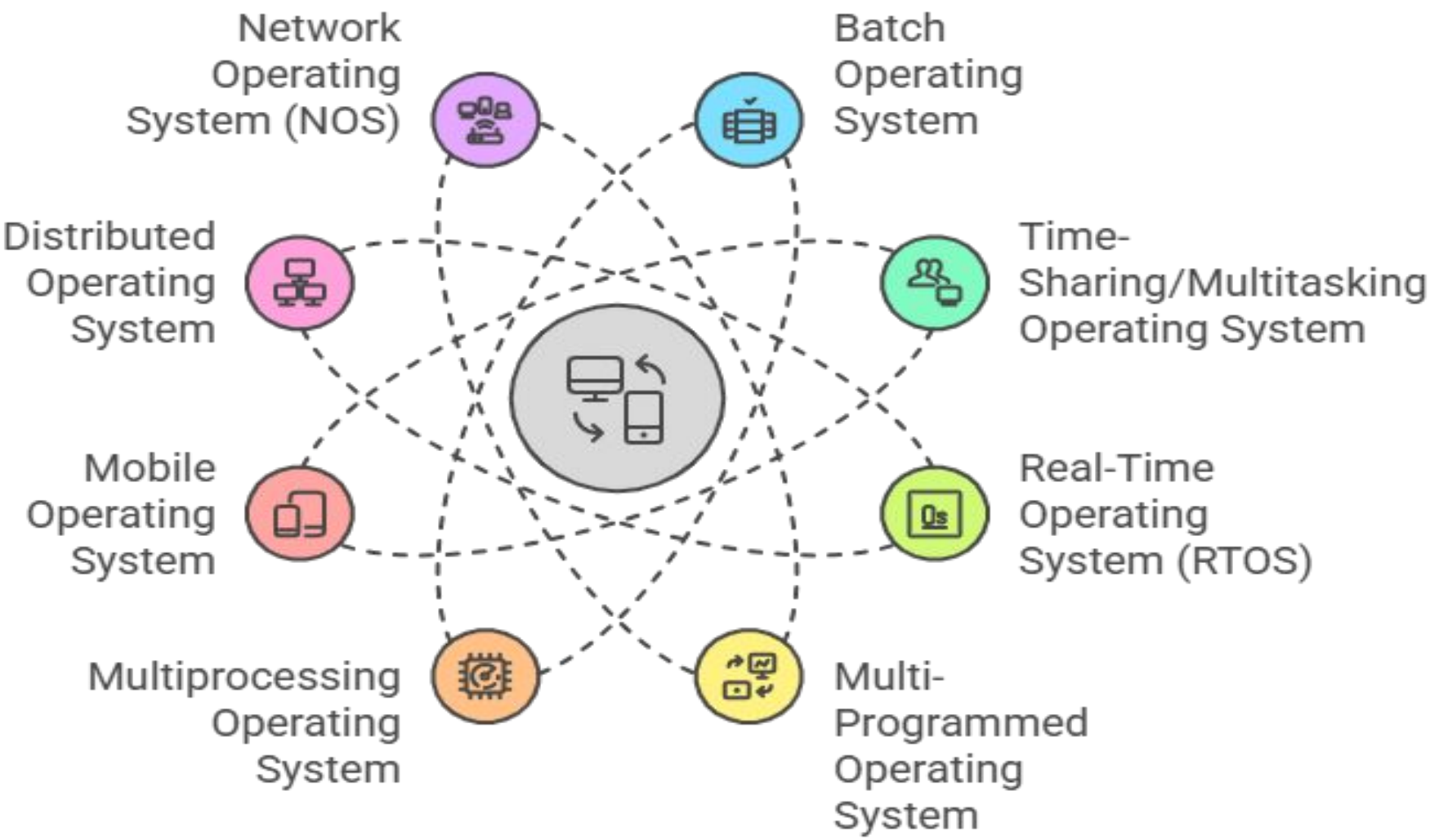
Fifth Generation (Present–Future): Networked, Distributed, and Mobile OS

- Cloud and Mobile OS dominate

- **Features:**
  - Multitasking, Multi-user, Security
  - Real-time OS, Embedded OS, Virtualization
  - IoT and AI integration

- **Examples**: Android, iOS, Windows 11, Linux, Chrome OS.

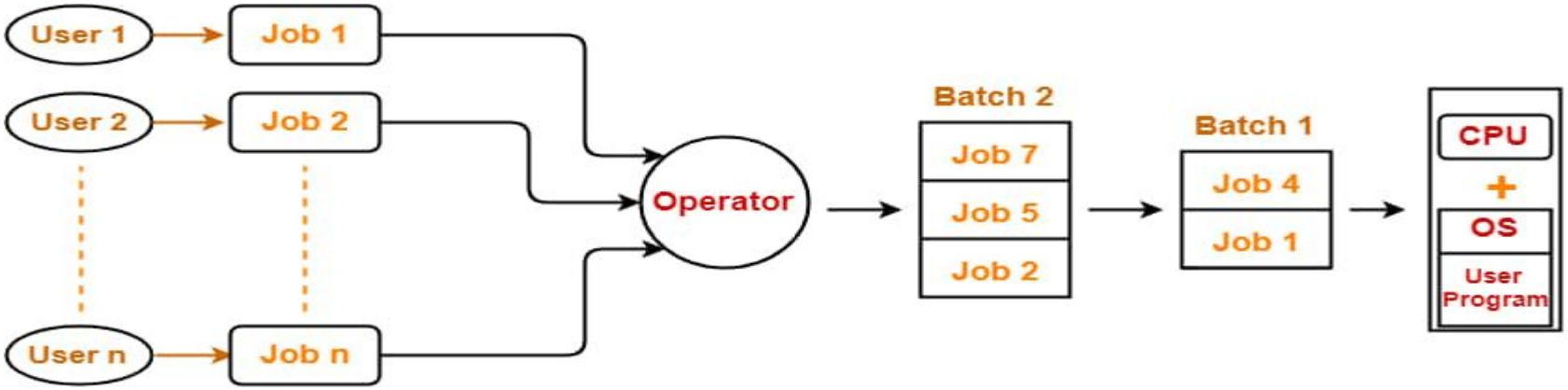
Types of Operating Systems



Made with Napkin

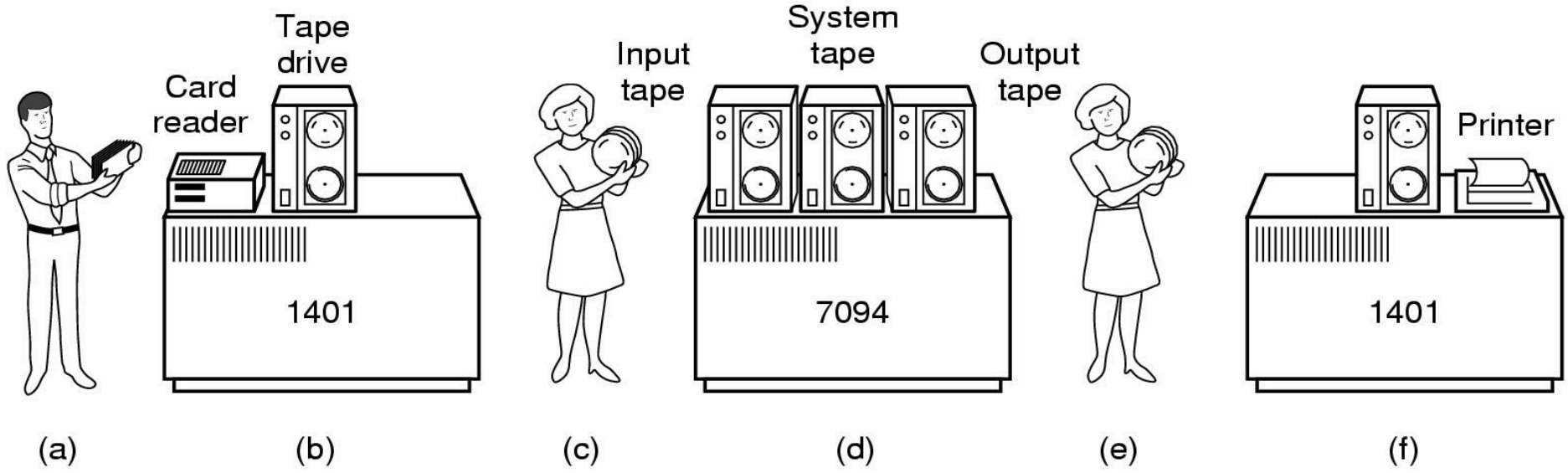
Batch Operating System

- In Batch processing same type of jobs *(BATCH- a set of jobs with similar needs)* batch (grouped) together and execute at a time.
- Executes a batch of jobs without user interaction.
- No real-time input/output.
- **Example**: IBM OS/360



Batch Operating System

Batch Operating System



Early Batch System

bring cards to 1401
read cards to tape
put tape on 7094 which does computing
put tape on 1401 which prints output

Batch Operating System

- Advantages –
  - The idle time batch system is very less
  - It is easy to manage large work repeatedly in batch systems

- Dis-Advantages –
  - **Sequential execution.**
  - **Starvation:** Different jobs in a single batch might take different amounts of time in execution.
  - **No interaction between job and user**

Time-Sharing/Multitasking Operating System

- Allows multiple users to share system resources simultaneously.

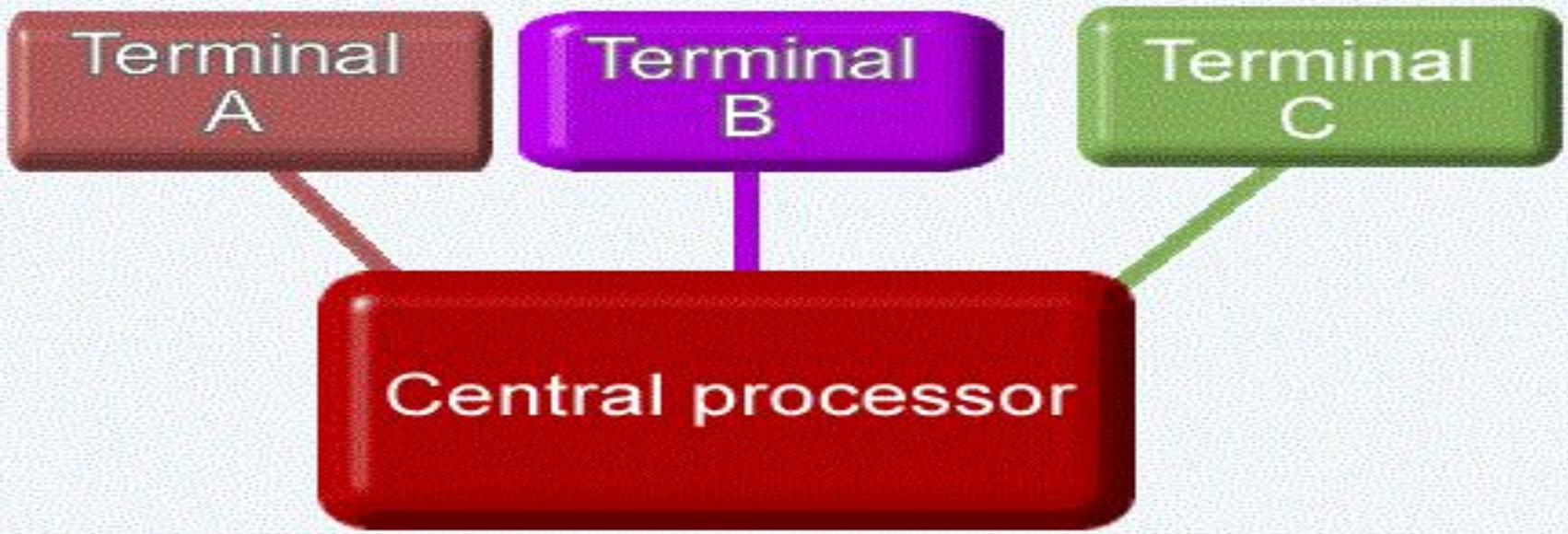
- In this, the CPU time is shared by different processes, so it is called as "Time sharing Systems".

- Time slice is defined by the OS, for sharing CPU time between processes.

- Quick response and interaction.

- **Example**: UNIX, MULTICS

Time-Sharing/Multitasking Operating System



Multiuser operating system share Processor time

Time Slices

Time-Sharing/Multitasking Operating System

Advantages –

- Data can be copied and moved between programs.
- Increases productivity, since many programs can be running at once.
- Any changes or updates are seen immediately. For example, if a new e-mail is received you immediately know.

Dis-Advantages –

- Requires more system resources.
- If on a Laptop or portable device takes more battery power.

Real-Time Operating System (RTOS)

- A system is said to be **Real Time** if it is required to complete it's work & deliver it's services on time.

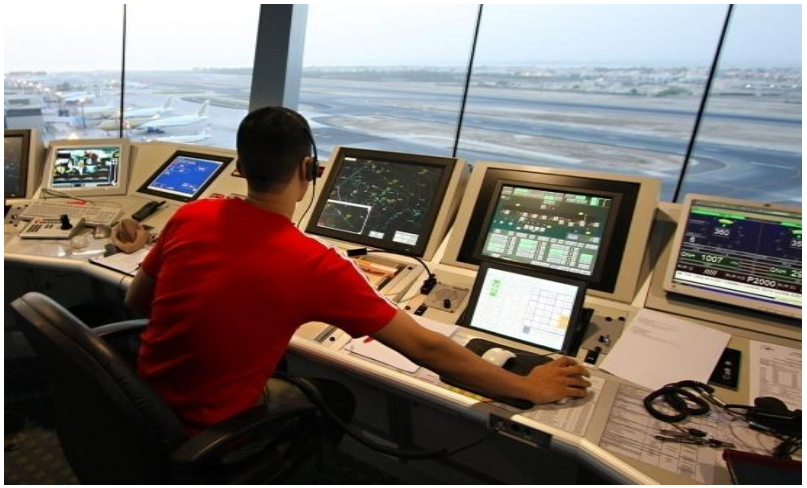
- It is defined as an operating system known to give maximum time for each of the critical operations that it performs, like OS calls and interrupt handling.

- **Example**: VxWorks, RTLinux

Real-Time Operating System (RTOS)

- **Hard Real Time System**
  - The Real-Time Operating system which guarantees the maximum time for critical operations and complete them on time are referred to as **Hard Real-Time Operating Systems.**

  - Example : Flight Control System, Pacemaker



Real-Time Operating System (RTOS)

- **Soft Real Time System**
  - While the real-time operating systems that can only guarantee a maximum of the time, i.e. the critical task will get priority over other tasks, but no assurity of completing it in a defined time.
  - These systems are referred to as **Soft Real-Time Operating Systems**.
  - Example :
    - Live video streaming,
    - Online Databases

Multiprogrammed Operating System

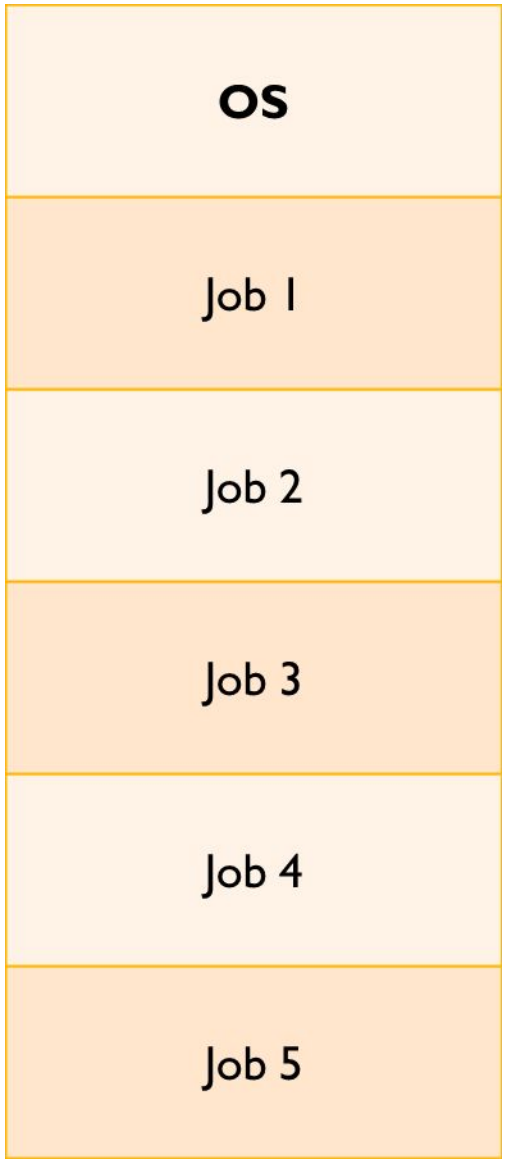
- Multiprogramming is a technique to execute number of programs simultaneously by a single processor.

- In Multiprogramming, number of processes reside in main memory at a time.

- The OS picks and begins to executes one of the jobs in the main memory.

- If any I/O wait happened in a process, then CPU switches from that job to another job. Hence CPU in not idle at any time.

Multi programmed Operating System



The main memory consists of 5 jobs at a time, the CPU executes one by one.

Advantages:

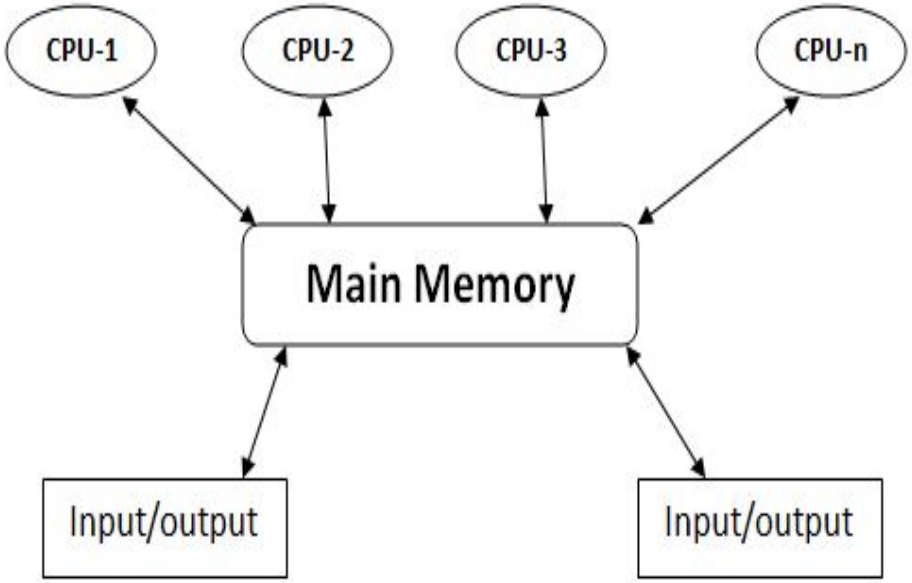
- Efficient memory utilization
- Throughput increases
- CPU is never idle, so performance increases.

Multiprocessing Operating System

- Supports two or more processors running simultaneously.

- Tightly coupled system – processors share memory and a clock; communication usually takes place through the shared memory.

- **Example**: Linux, Window



Multiprocessing Operating System

- **Symmetric multiprocessing (SMP)**
  - Each processor runs an identical copy of the operating system.
  - Many processes can run at once without performance drop.
  - Most modern operating systems support SMP

- **Asymmetric multiprocessing**
  - Each processor is assigned a specific task; master processor schedules and allocates work to slave processors.
  - More common in extremely large systems

Multiprocessing Operating System

- Advantages –
  - Increased throughput
  - Economical
  - Increased reliability

- Dis-Advantages –
  - If one processor fails then it will affect in the speed

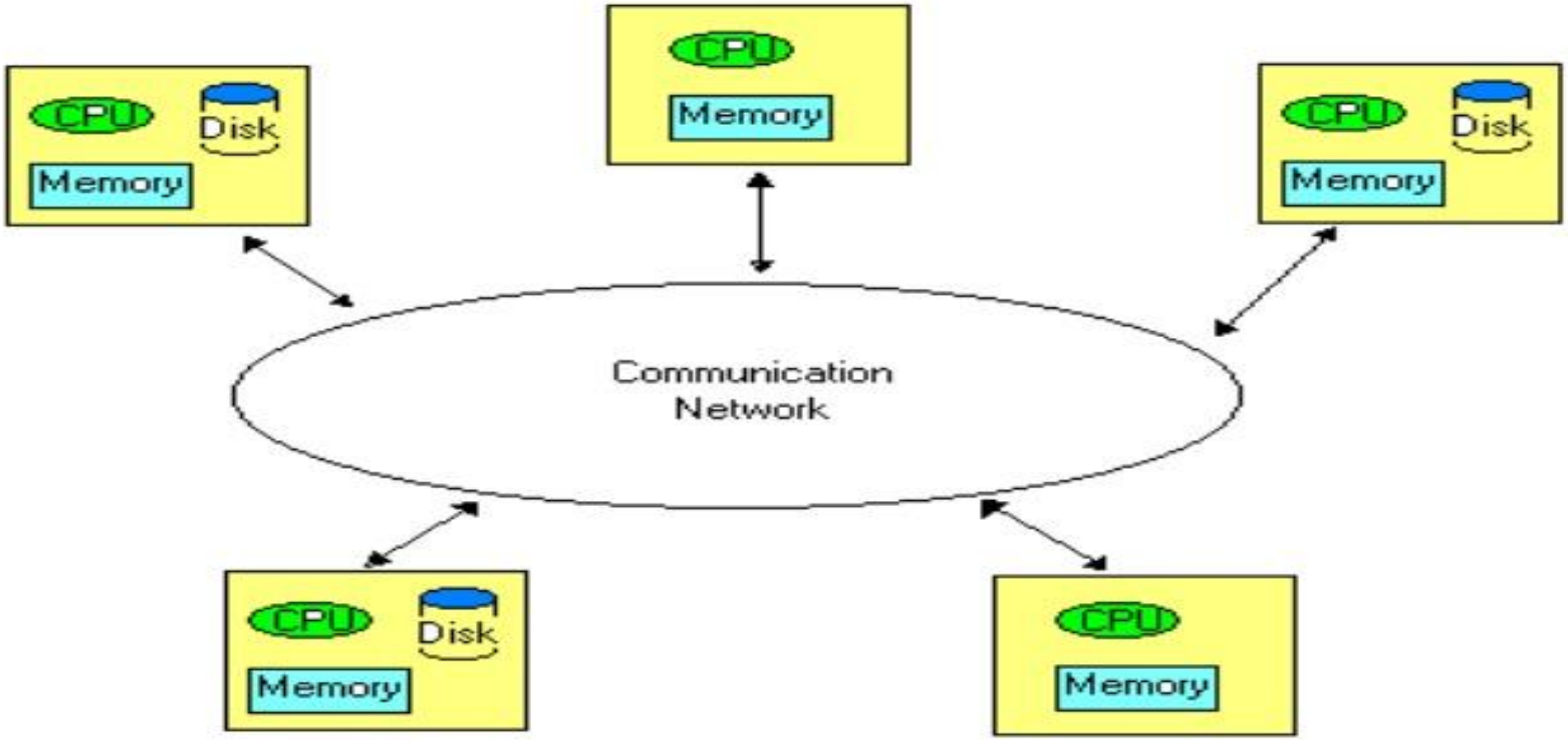
Distributed Operating System

- Distribute the computation among several physical processors.

- **Manages a group of independent computers as a single system.**

- Each processor has its own local memory, processors communicate with one another through various communications lines, such as high-speed buses or telephone lines.

Architecture of Distributed OS



Distributed Operating System

Advantages –

- With resource sharing facility, a user at one site may be able to use the resources available at another.

- Speedup the exchange of data with one another via electronic mail.
- If one site fails in a distributed system, the remaining sites can potentially continue operating.

- Reduction of the load on the host computer.
- Reduction of delays in data processing.

Dis-Advantages –

- Security problem due to sharing.
- Some messages can be lost in the network system.
- If there is a database connected on local system and many users accessing that database through remote or distributed way then performance become slow

Mobile Operating System

- Designed for mobile devices with touch interfaces.

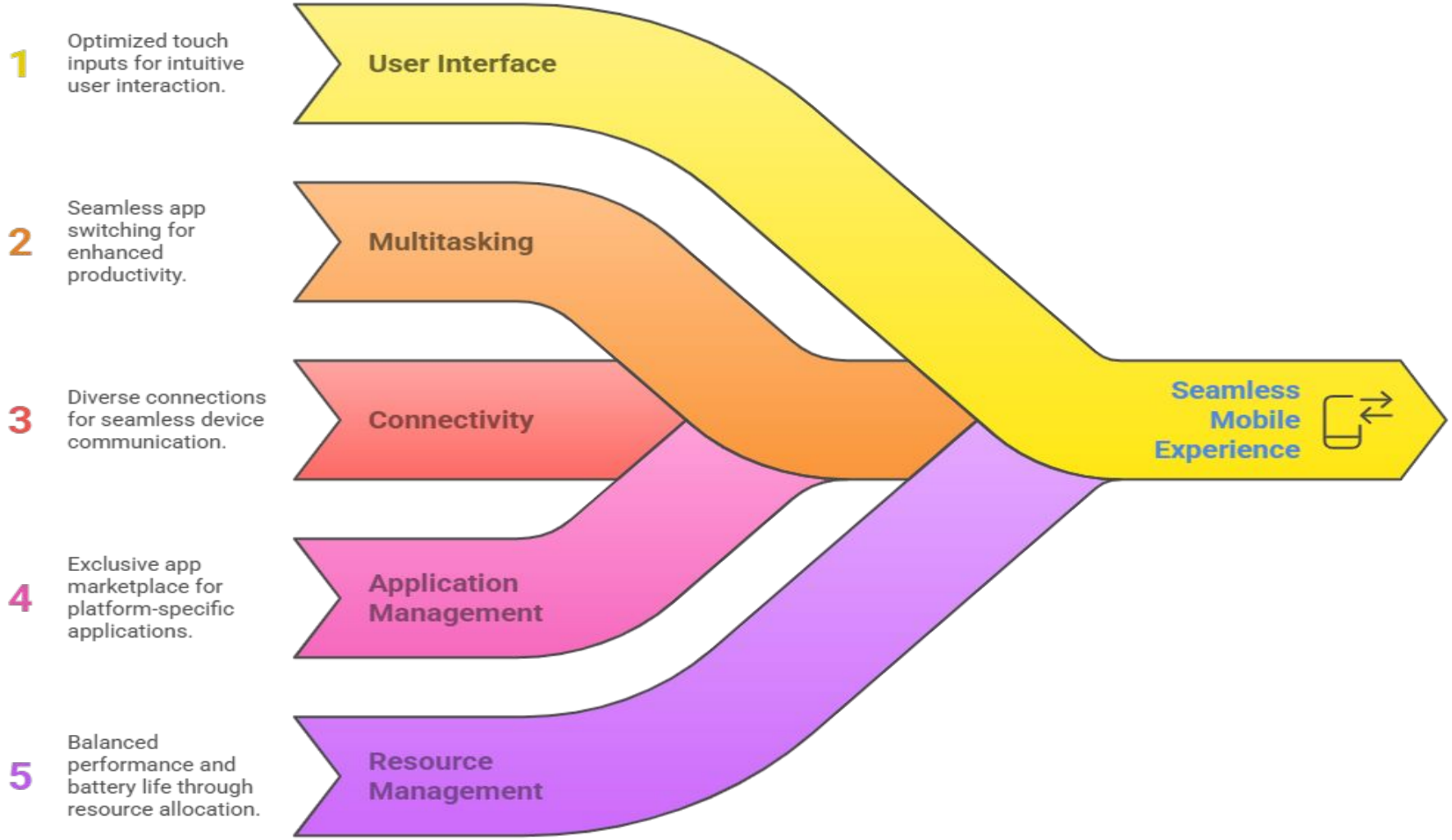
- It is Lightweight and It runs apps, controls memory, and connects to networks. These systems provide an easy-to-use interface..

- Optimized for battery and sensors.

- **Examples**: Android, iOS

Mobile Operating System

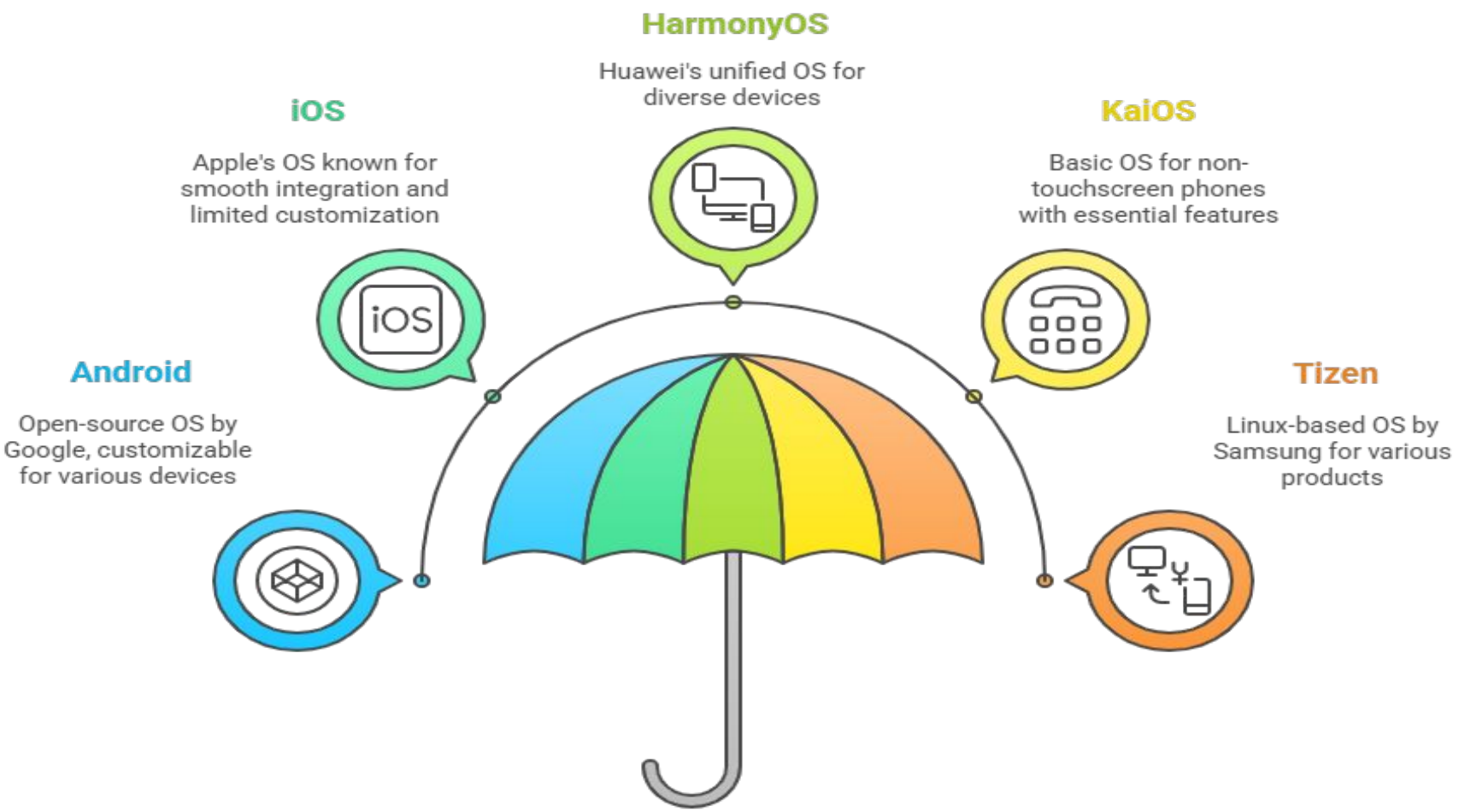
Mobile OS Features



Made with Napkin

Mobile Operating System

Types of Mobile Operating Systems



Made with Napkin

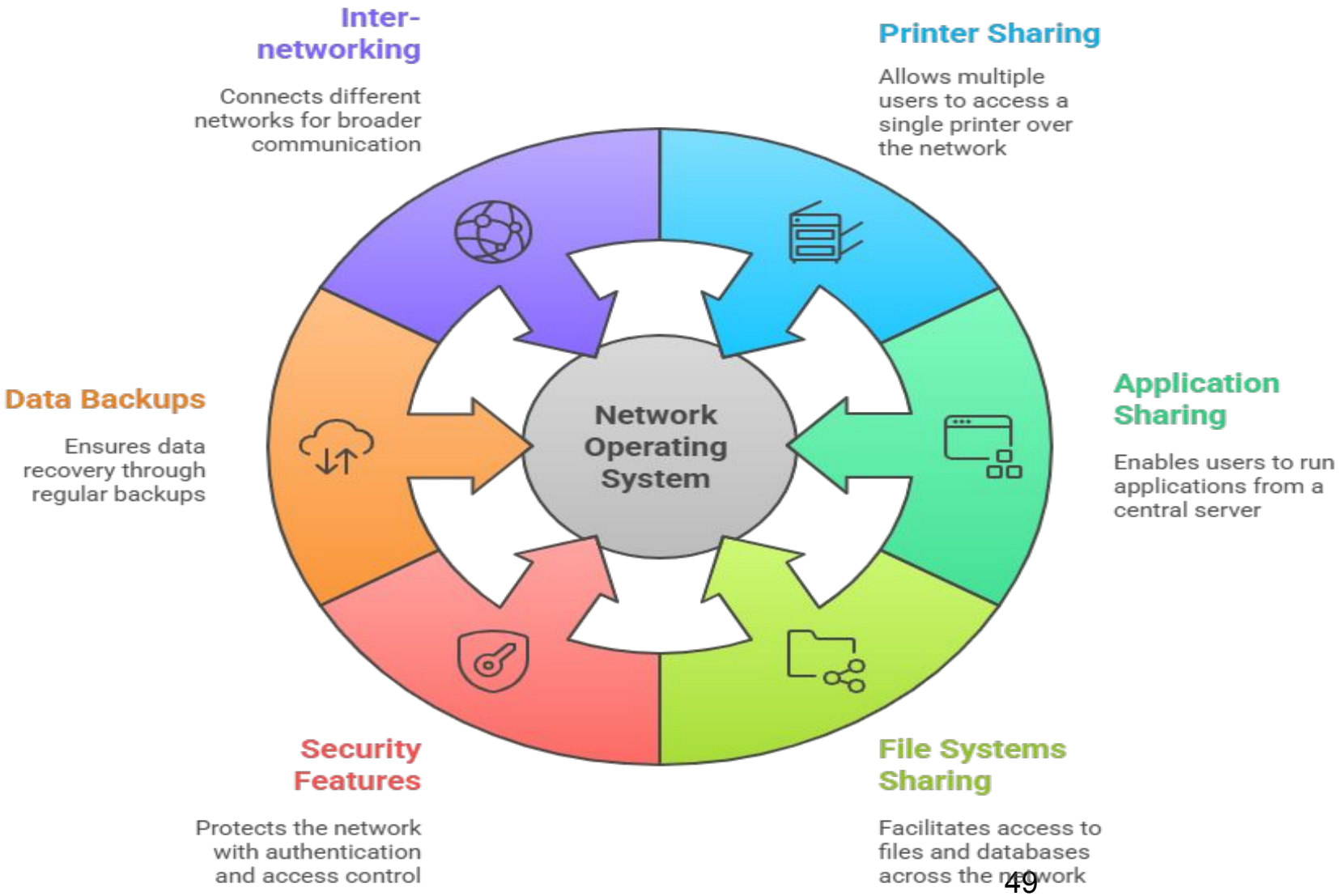
Network Operating System (NOS)

- Network operating system(NOS) is software that connects multiple devices and computers on the network and allows them to share resources on the network.

- **Functions of the NOS (Network Operating System)**
  - Creating and managing user accounts on the network.
  - Controlling access to resources on the network.
  - Provide communication services between the devices on the network.
  - Monitor and troubleshoot the network.
  - Configuring and Managing the resources on the network.

Network Operating System (NOS)

Features of Network Operating Systems



Made with Napkin

Network Operating System (NOS)

- Advantages –
  - Highly stable due to central server.
  - Provide good security.
  - Up-gradation of new technology and hardware can be easily implemented in the network.
  - Provide remote access to servers from different locations.

Network Operating System (NOS)

- Dis-Advantages –
  - Depend on the central location to perform the operations.
  - High cost to buying server.
  - Regular updating and maintenance are required.

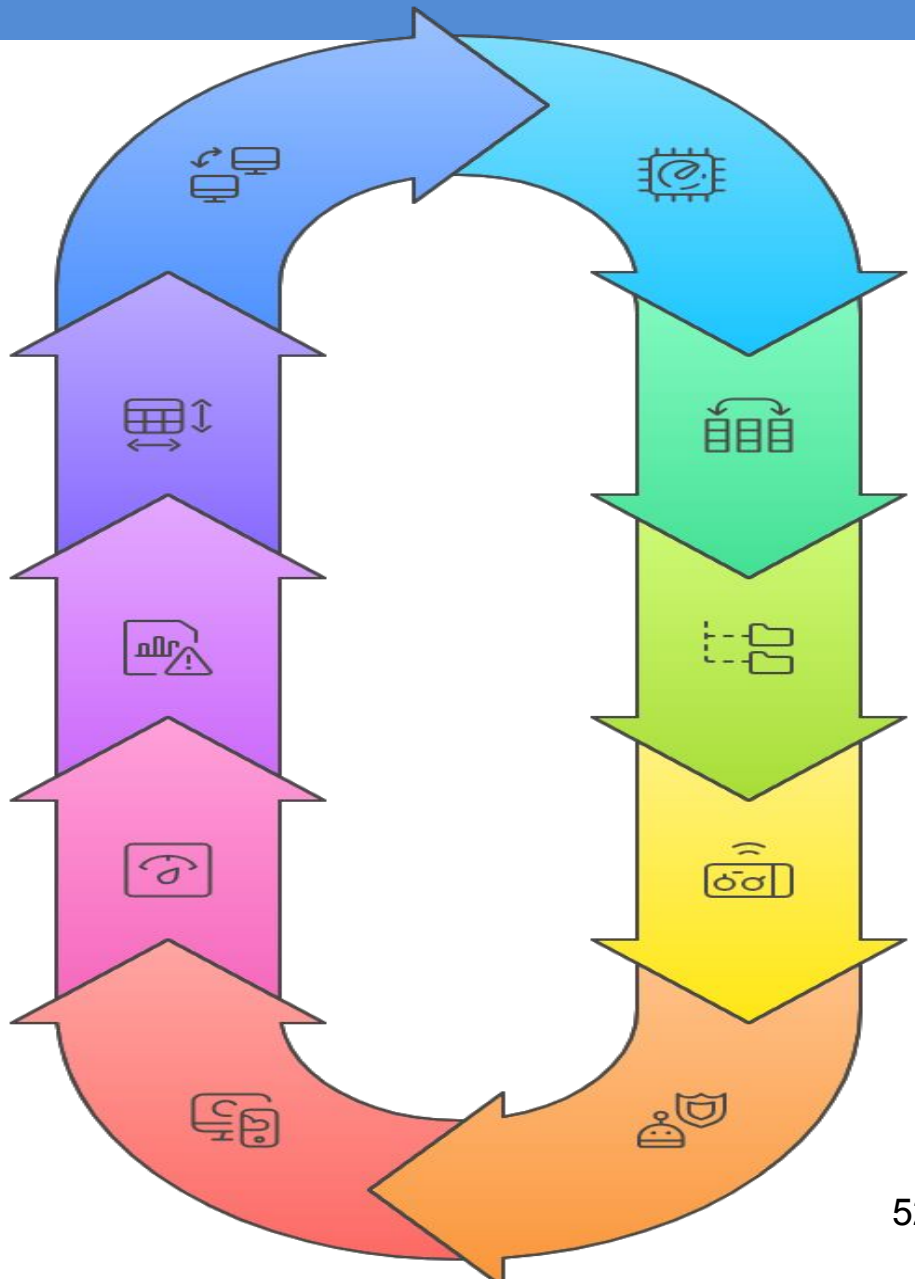
- **Examples of NOS**

❖ Microsoft Windows Server

❖ UNIX/Linux

❖ Artisoft's LANtastic

OS Services



Networking Services

Facilitate network communication

Process Management

Efficiently handle running applications

Resource Allocation

Distribute resources effectively

Memory Management

Optimize memory usage

Error Detection and Handling

Identify and resolve issues

File System Management

Organize and access files

System Performance Monitoring

Track system efficiency

Device Management

Control hardware interactions

User Interface Services

Enhance user experience

Security and Protection

Safeguard system integrity

OS Services

1. Process Management

- **Creation and deletion** of user and system processes.
- **Scheduling** of processes for efficient CPU usage
- **Synchronization** and **communication** between processes (e.g., IPC)
- **Deadlock handling**
- ➡ Example: Running multiple apps (like a browser and a media player) at the same time.

OS Services

2. Memory Management

- Keeps track of **each byte of memory** and its status (free or allocated)

- **Allocates/de-allocates memory** to processes when needed

- Implements memory techniques like **paging**, **segmentation**, and **virtual memory**

- ➡ Example: Ensuring your program has enough memory space to run.

OS Services

3. File System Management

- Organizes, stores, retrieves, names, shares, and **protects files**

- Provides support for **file operations**: read, write, open, close, delete, etc.

- Maintains file and directory structure.

- ➡ Example: Saving and opening documents from your file explorer.

OS Services

4. Device Management

- Manages **I/O devices** (keyboard, mouse, disk drives, printers, etc.)

- Uses **device drivers** for communication between devices and the OS.

- Implements buffering, caching, and spooling.

- ➡ Example: Sending a document to the printer while using other programs.

OS Services

5. Security and Protection

- Protects **data and resources** from unauthorized access.

- Uses techniques like **authentication (login/password)**, **access control lists**, and **encryption.**

- Ensures safe execution of processes.

- ➡ Example: Preventing malware from accessing system files.

6. User Interface Services

- Provides **User Interfaces (UIs)**:
  - **Command Line Interface (CLI)** – e.g., Terminal, Command Prompt
  - **Graphical User Interface (GUI)** – e.g., Windows Desktop, macOS

- ➡ Example: Using Windows Explorer to browse files or Terminal to run commands.

OS Services

7. Networking Services

- Enables communication between computers via networks (LAN, WAN, Internet)

- Implements protocols like **TCP/IP**, **FTP**, **HTTP**, etc.

- ➡ Example: Connecting to the internet or sharing files between computers.

OS Services

8. System Performance Monitoring

- Tracks system performance and **resource utilization.**

- Helps detect bottlenecks, optimize usage, and support **system tuning.**

- ➡ Example: Task Manager showing CPU, memory, and disk usage.

9. Error Detection and Handling

- Detects various hardware and software errors.

- Ensures the system responds to errors gracefully (without crashing)

- ➡ Example: Displaying a message when a program crashes or a file is corrupted.

OS Services

10. Resource Allocation

- Allocates system resources (CPU, memory, I/O) to various programs efficiently.

- Ensures fair and optimized utilization of resources.

- ➡ Example: Ensuring that both a video player and antivirus can run without interference.

System Call

- A **System Call** is a **programmatic way for a user-level process to request a service** from the **operating system kernel**.

- Since user applications cannot directly access hardware or critical OS functions, they use system calls to interact with the system securely.
- Act as the **interface** between **user programs** and the **OS kernel**

- **Example:** When you open a file using fopen() in C, it internally uses the open() system call.

Step wise Procedure for System Call Execution

User Program Request

A user program initiates a system call by invoking a specific function.

Parameter Passing

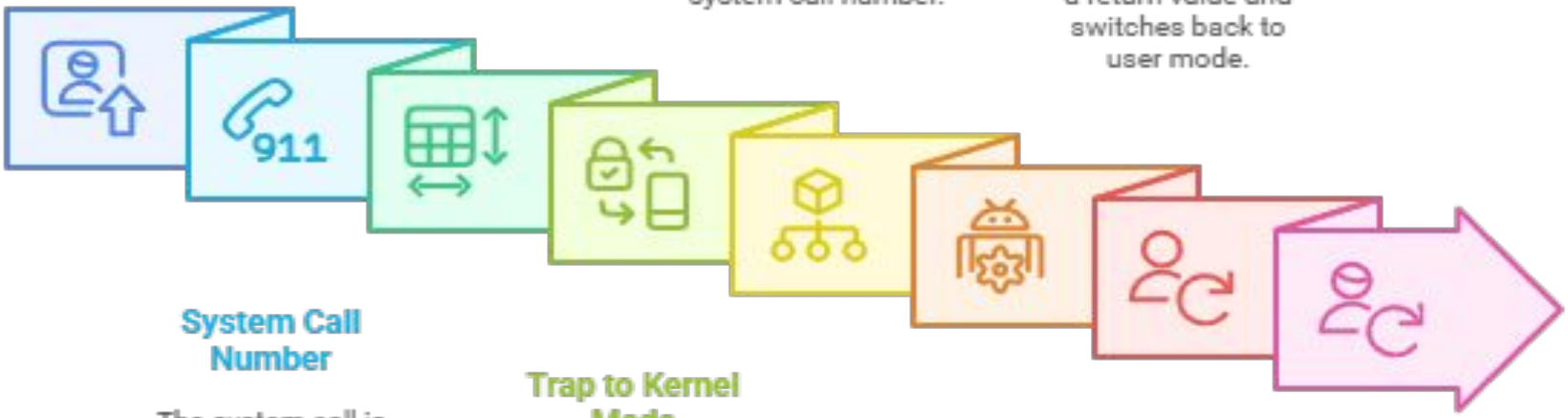
Parameters are passed to the system call through registers or the stack.

Kernel Handling

The kernel identifies the appropriate function to execute based on the system call number.

Return to User Mode

The kernel prepares a return value and switches back to user mode.



System Call Number

The system call is identified by a unique number.

Trap to Kernel Mode

A special instruction switches the program from user mode to kernel mode.

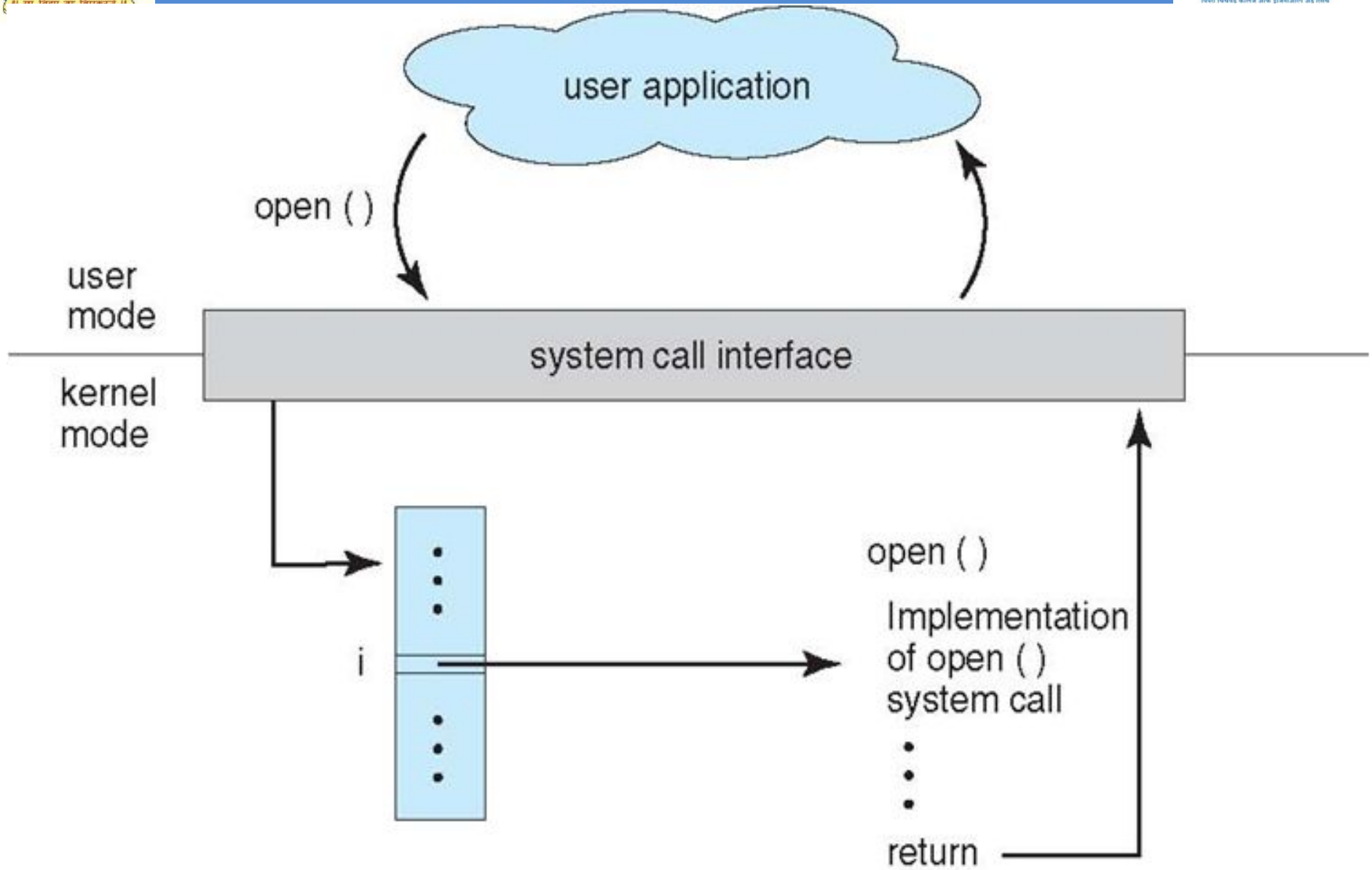
Service Execution

The kernel executes the requested service using the provided parameters.

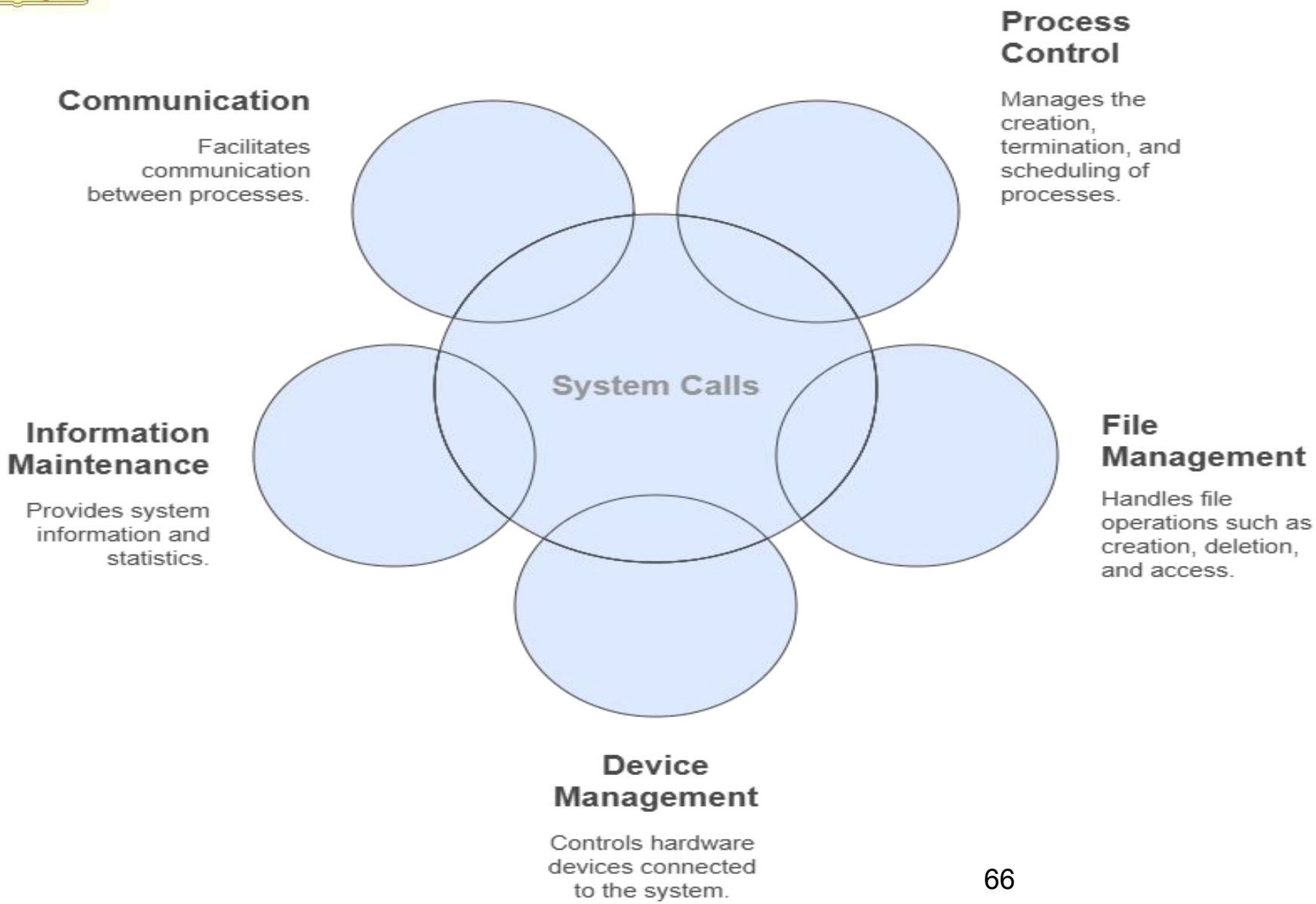
Return to User Program

The user program receives the return value and continues execution.

How System Call Work



Types of System Call



Types of System Call

1. Process Control

- Used to create, terminate, or manage processes.

Common System Calls	Description
fork()	Create a new process
exec()	Load a new program into process
exit()	Terminate process execution
wait()	Wait for a process to finish
getpid()	Get process ID
kill()	Send a signal to a process

Types of System Call

2. File Management

- Used to perform operations on files and directories.

Common System Calls	Description
open()	Open or create a file
read()	Read from a file
write()	Write to a file
close()	Close a file
unlink()	Delete a file
lseek()	Reposition read/write file offset

Types of System Call

3. Device Management

- Used to request and release hardware resources.

Common System Calls	Description
ioctl()	Device-specific input/output ops
read() / write()	Transfer data to/from device
open() / close()	Access or release a device

Types of System Call

4. Information Maintenance

- Used to get/set system information.

Common System Calls	Description
gettimeofday()	Get current time
settimeofday()	Set current time
getuid()	Get user ID
getgid()	Get group ID

Types of System Call

5. Communication (Inter-process Communication - IPC)

- Used for message passing and synchronization between processes.

Common System Calls	Description
pipe()	Create a pipe
shmget()	Allocate shared memory
mmap()	Map files or devices into memory
msgsnd() / msgrcv()	Send/receive message queues
semget()	Semaphore operations(Signaling Mechanism)
socket()	Create network socket

Structure of Operating System

- The **structure of an operating system (OS)** refers to the way its components are organized and how they interact to manage hardware and software resources efficiently.

- Different structures are used depending on design goals like performance, flexibility, simplicity and security.

- Because operating systems have complex structures, we want a structure that is easy to understand so that we can adapt an operating system to meet our specific needs.

Main Objectives of OS Structure:

- Simplify development
- Manage complexity
- Ensure reliability and efficiency
- Provide modularity

Types of Operating System Structure

Emulates hardware environments

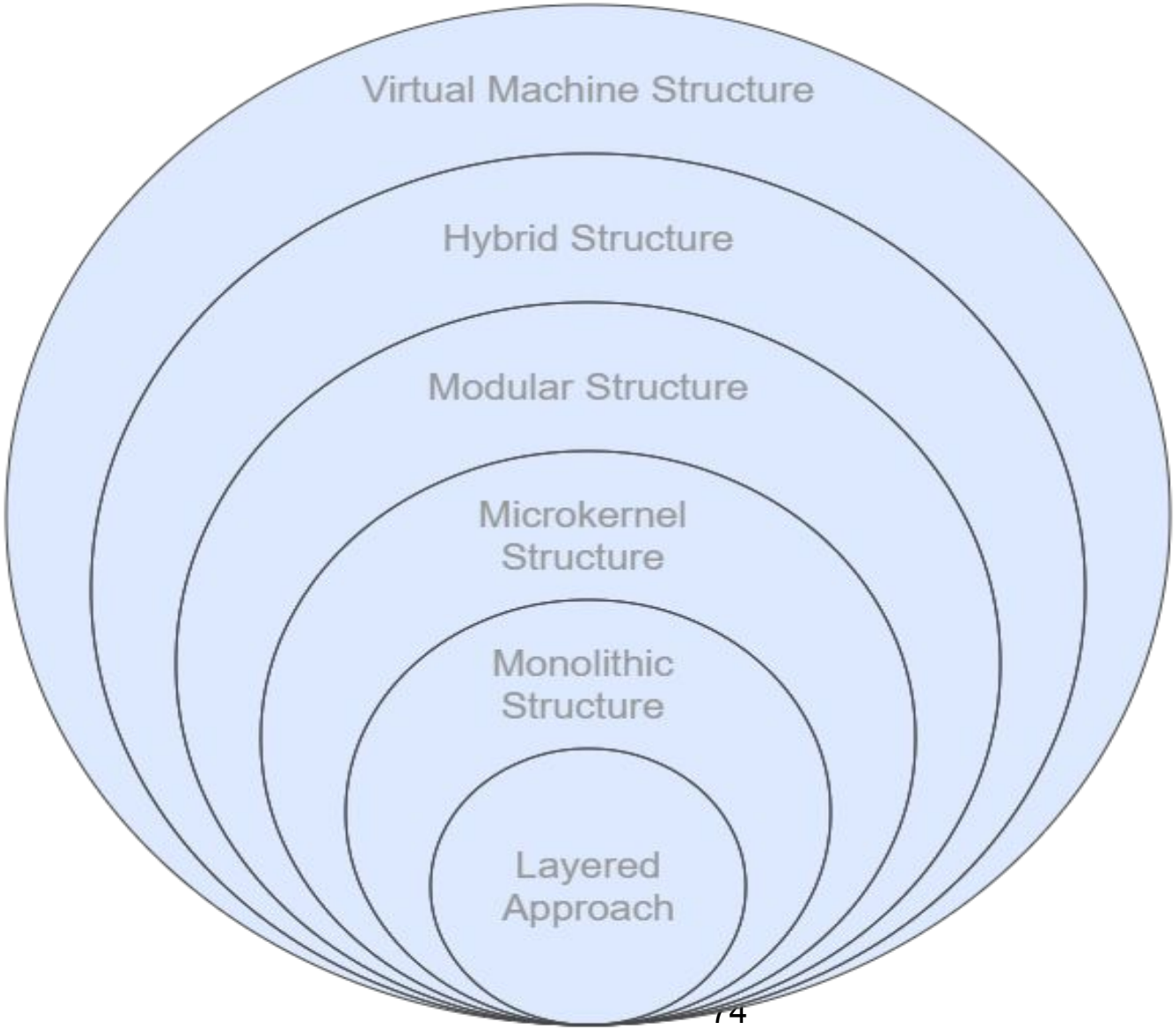
Combines different architecture types

Independent, interchangeable modules

Core functionality with modules

Single, unified codebase

Simplifies system design with layers



Made with Napkin

Layered Structure of Operating System

- The modularization of a system can be done in many ways.

- One method is the layered approach, in which the operating system is broken up into a number of layers (or levels), each built on top of lower layers.

- The bottom layer (layer 0) is the hardware; the highest (layer N) is the user interface.

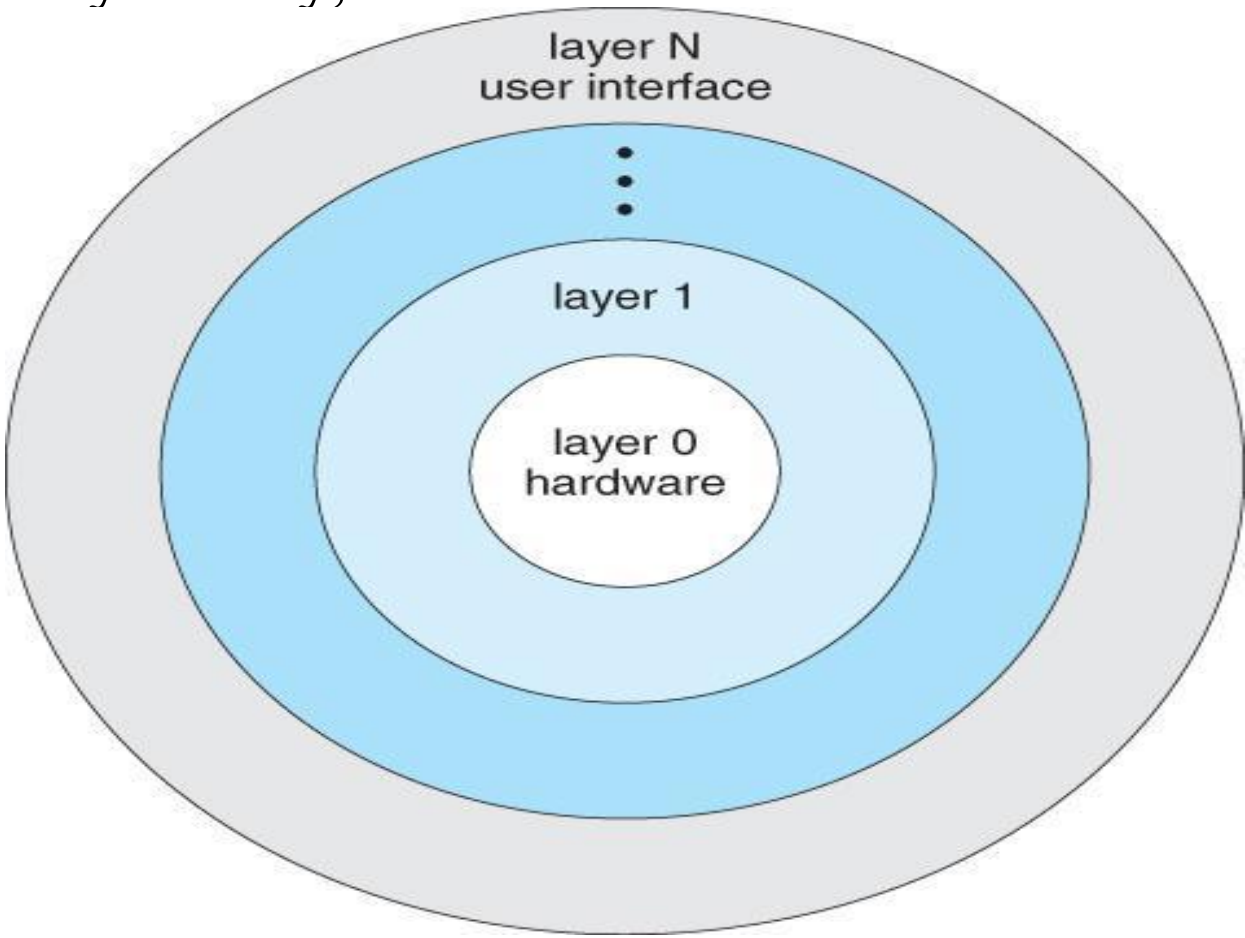
- These layers are so designed that each layer uses the functions of the lower-level layers.

Layered Structure of Operating System

This simplifies the debugging process, if lower-level layers are debugged and an error occurs during debugging, then the error must be on that layer only,

as the lower-level layers have already been debugged.

UNIX is an example of this structure.



Layered Structure of Operating System

Advantages:

- Modular and easy to debug or modify.
- Provides **abstraction** between hardware and software.

Dis-Advantages:

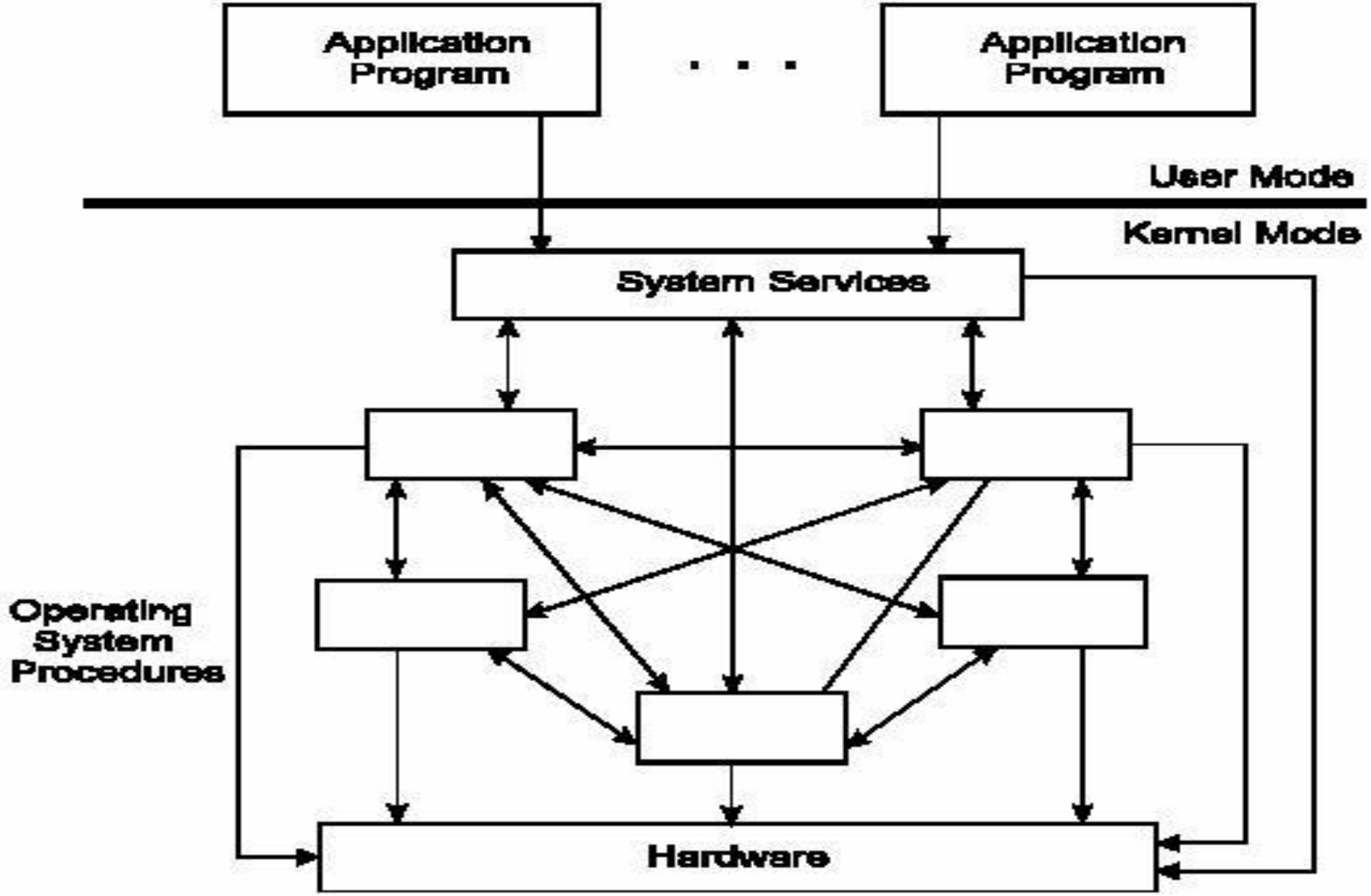
- Less efficient due to **overhead** between layers.
- It requires **careful planning** for designing the layers, as the higher layers use the functionalities of only the lower layers.

Monolithic Structure of Operating System

- A monolithic structure is a type of operating system architecture where the entire operating system is implemented as a single large process in kernel mode.

- Essential operating system services, such as process management, memory management, file systems, and device drivers, are combined into a single code block.

Monolithic Structure of Operating System



Monolithic Structure of Operating System

- Operating system distinguishes between system mode and user mode while executing an application program.

- An application program runs in the user mode.

- A user makes a request for a service using application programs.

- The operating system locates a system call and executes it in the system mode.

Monolithic Structure of Operating System

Advantages:

- **High performance** due to direct communication between components.
- It is **easier to build** because all parts are in one code block.

Dis-Advantages:

- **Hard** to maintain or debug.
- **One bug** can crash the entire system.

Example: Early UNIX, MS-DOS

Microkernel Structure of Operating System

- Micro-Kernel structure designs the operating system by removing all non-essential components from the kernel and implementing them as system and user programs.

- This results in a smaller kernel called the micro-kernel.

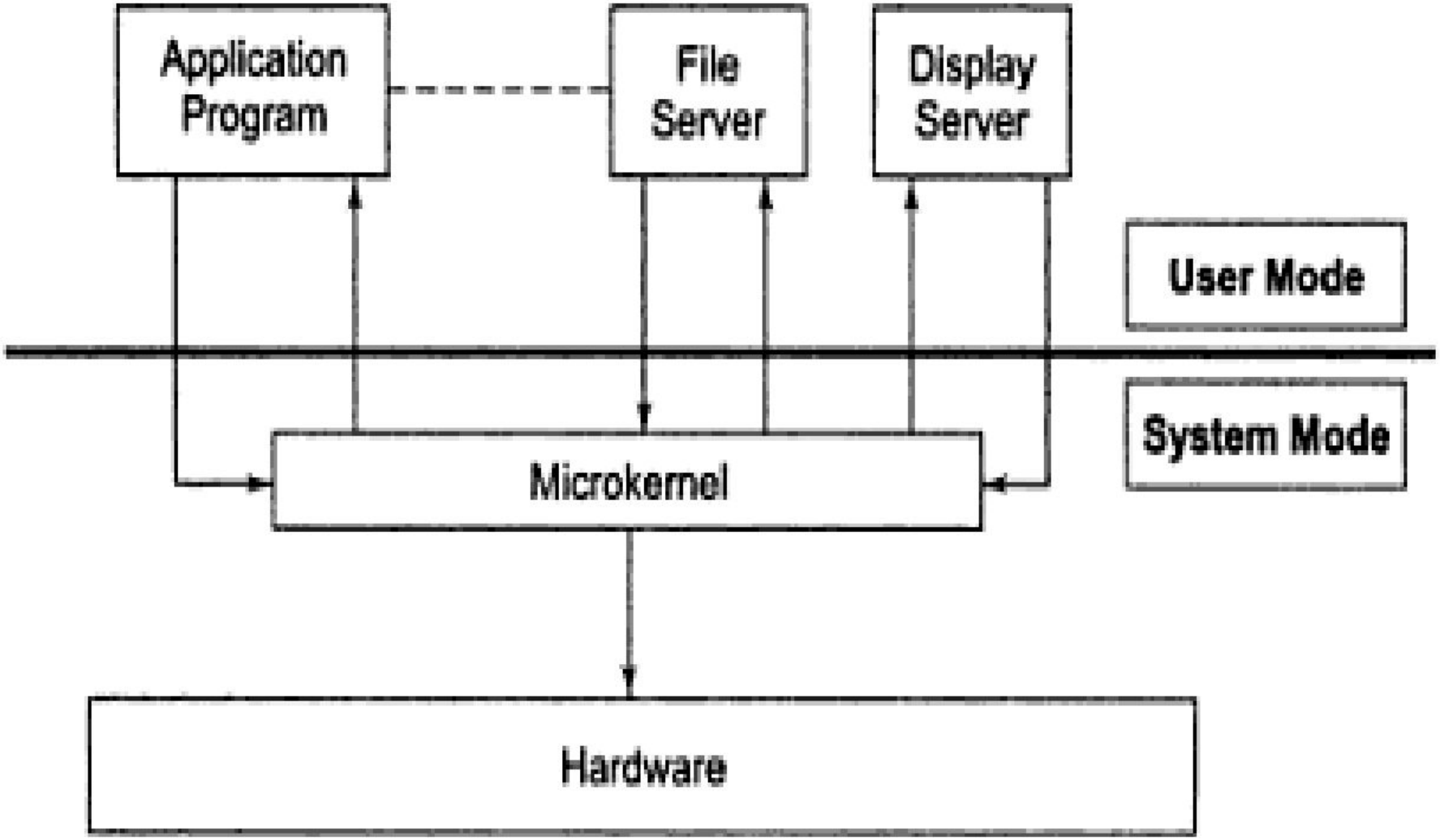
- In this system, kernel provides only the most essential operating system functions like

  process management,

  communication primitives and

  low level memory management.

Microkernel Structure of Operating System



Microkernel Structure of Operating System

- Advantages of this structure are that all new services need to be added to user space and does not require the kernel to be modified.

- Thus it is more secure and reliable as if a service fails, then rest of the operating system remains untouched.

📌 Mac OS is an example of this type of OS.

Microkernel Structure of Operating System

Advantages:

- More secure and stable.
- Easy to extend and modify.

Dis-Advantages:

- Performance can be slower due to inter-process communication (IPC).

Introduction to Linux Operating System(Monolithic Structure)

- Linux is a powerful, open-source operating system (OS) that forms the backbone of countless systems worldwide, including servers, desktops, smartphones, embedded systems, and supercomputers.

- It is known for its stability, security, flexibility, and community-driven development model.

- Linux is a **Unix-like** operating system based on the **Linux kernel**, originally developed by **Linus Torvalds** in 1991.

Key Features of Linux OS

Stability

Runs without needing a reboot for long periods

Open Source

Free to use, modify, and distribute

Portability

Runs on a wide variety of hardware

Multitasking

Handles multiple tasks simultaneously

Security

Robust user permissions and updates

Multiuser

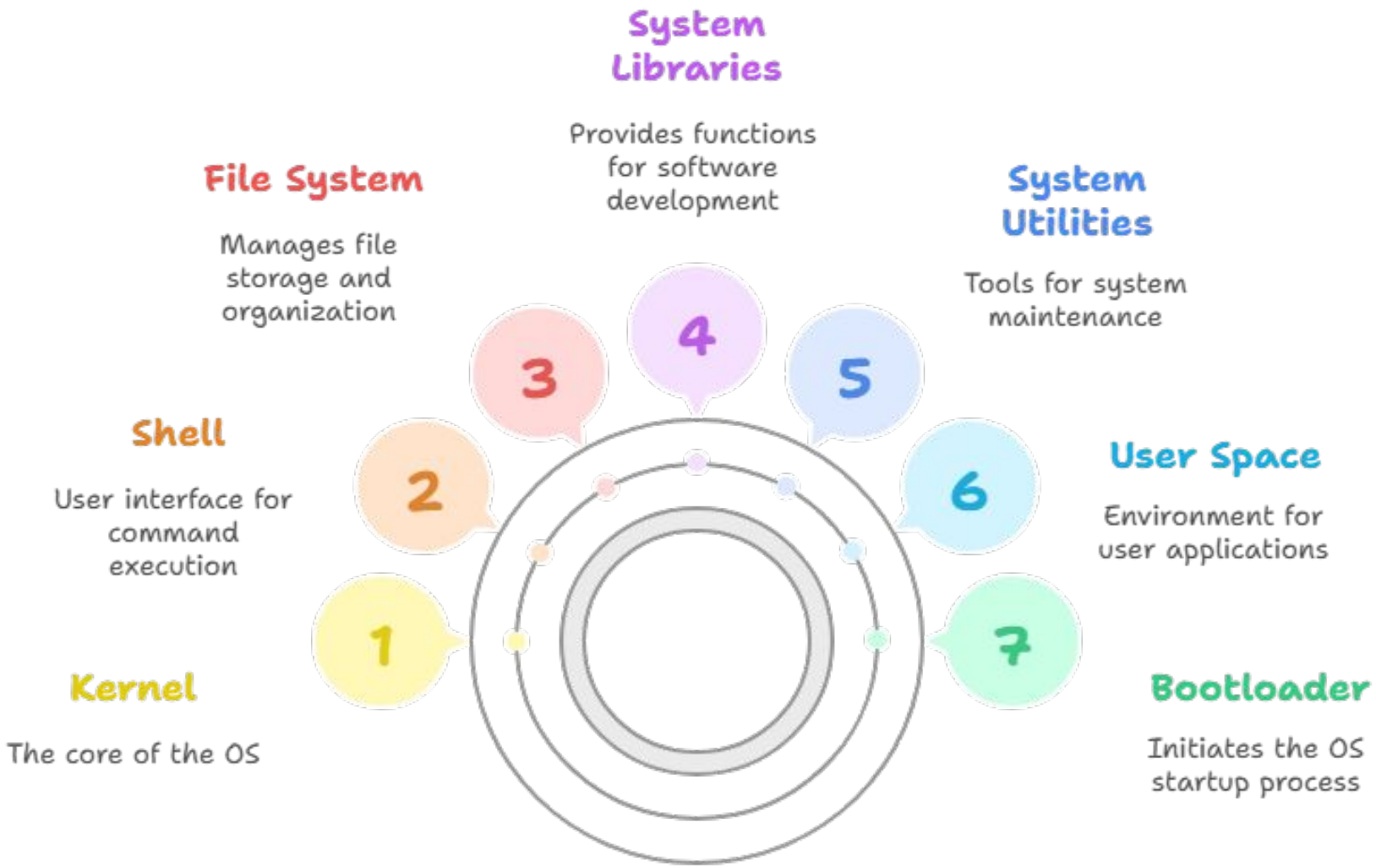
Supports multiple users at the same time

Made with Napkin

Components of Linux system

- The Linux operating system is composed of several key components that work together to manage system resources and provide functionality to users and applications.

Components of Linux system



Made with Napkin

Components of Linux system - 🔧 1. Kernel

- **Core part** of the Linux operating system.
- Manages all system resources: **CPU**, **memory**, **I/O devices**, etc.
- Provides a layer between hardware and applications.
- Types of kernels in Linux: **Monolithic** (e.g., traditional Linux kernel), **Microkernel**, etc.

Components of Linux system - 2. Shell

- Acts as a **command-line interface (CLI)** between the user and the kernel.

- Interprets user commands and executes them using the kernel.

- **Types of Shells:**
- **Bash (Bourne Again SHell)** – default in most distributions.

- **sh**, **csh**, **ksh**, **zsh** – alternatives with different features and syntax.

Components of Linux system - 3. File System

- Organizes data in a hierarchical structure (tree-like format).

- Everything in Linux is treated as a **file** (including devices and processes).

- **Key directories:**

  / – Root directory

  /home – User directories

  /etc – Configuration files

  /bin, /sbin – Essential system binaries

  /dev, /proc – Devices and process info

Components of Linux system - 4. System Libraries

- Special functions and routines used by applications and the OS.
- Help applications interact with the kernel without needing to know kernel details.
- Example: **Glibc** (GNU C Library)

Components of Linux system - 5. System Utilities

- Essential tools and programs that perform system-related tasks.
- Include both basic (e.g., ls, cp, mv, top) and advanced utilities.

- **Categories:**
  - File management: rm, mv, cp, find
  - Process management: ps, kill, top
  - Disk management: df, du, mount, fdisk
  - Networking: ping, netstat, ifconfig, ssh

Components of Linux system - 6. User Space

- Where user applications run.

- Includes:

✔ **User applications** (editors, browsers, compilers)

✔ **Desktop environments** (GNOME, KDE, XFCE)

✔ **Graphical User Interface (GUI)** tools

Components of Linux system - 7. Bootloader

- A program that **loads the Linux kernel** into memory and starts the OS.

- Examples: **GRUB**, **LILO**

- Initializes the system and hands over control to the kernel.

Basic Shell Commands

📁 File and Directory Commands

Command	Description	Example
ls	List files and directories	ls -l (detailed list)
pwd	Show current directory path	pwd
cd	Change directory	cd /home/user
mkdir	Create a new directory	mkdir new_folder
rmdir	Remove an empty directory	rmdir old_folder
touch	Create a new empty file	touch file.txt
rm	Remove file or directory	rm file.txt, rm -r folder/
cp	Copy files or directories	cp a.txt b.txt, cp -r dir1/ dir2/
mv	Move or rename files	mv old.txt new.txt

Basic Shell Commands

File Viewing and Editing

Command	Description	Example
cat	Display file contents	cat file.txt
more, less	View file page by page	less longfile.txt
head	Show first 10 lines	head file.txt
tail	Show last 10 lines	tail file.txt
nano, vi, vim	Text editors in the terminal	nano file.txt, vi file.txt

Basic Shell Commands

File Permissions and Ownership

Command	Description	Example
chmod	Change file permissions	chmod 755 file.sh
chown	Change file owner	chown user:group file.txt

Basic Shell Commands

🔍 Searching

Command	Description	Example
find	Search files/directories	find . -name "*.txt“
grep	Search text in files	grep "hello" file.txt

Basic Shell Commands

🧠 Process Management

Command	Description	Example
ps	Show running processes	ps aux
top	Live process monitor	top
kill	Terminate a process by PID	kill 1234
killall	Kill processes by name	killall firefox

Basic Shell Commands

📶 Networking

Command	Description	Example
ping	Check network connection	ping google.com
ifconfig / ip a	Display network interfaces	ip a
netstat	Show network connections	netstat -tuln

Basic Shell Commands

System Information and Cleanup

Command	Description	Example
df	Show disk space usage	df -h
du	Show file/directory size	du -sh *
free	Show memory usage	free -m
clear	Clear the terminal screen	clear
history	Show command history	history

Basic Shell Commands

Package Management

Command	Description	Example
apt update	Refresh package list	sudo apt update
apt install	Install a package	sudo apt install vim
apt remove	Remove a package	sudo apt remove vim

Thank You